

סיכום שיעור 12

ליה גולדינג

השיעור התחלק לשני חלקים - מעבר על פונקציית window, ולאחר מכן דיון על בחירת המודלים לפרוייקט.

חלק 1 - Window Functions

מטרת השימוש בחלון

שאלות רגילות מאפשרות לנו לראות ולבצע חישובים על הרשומות הגולמיות.

שאלות group by מאפשרות לנו לחלק את הרשומות לקבוצות שוות במאפיינים מסויימים ולשלוף אגרגציות על הקבוצה.

שאלות חלון מאפשרות לרשומה גולמית "להציץ" על חלק מהרשומות הנשלפות איתה, לדוגמה כי להשוות בינה לאחרות.

חלונות והגדרתם

אנחנו מגדירים לכל רשומה חלון ובו הרשומות עליהן אנו רוצים להציץ.

נגדיר את החלון על ידי הפרמטרים הבאים-

- **Current record**

פרמטר זה משתמע ולא מופיע ישירות אולם לכל רשומה חלון משלה.

- **Name**

שם לחלון על מנת שנוכל להשתמש בו שוב וכדי להבהיר את משמעותו

- **Partition**

השדות לפיהם נקבע מי הרשומות שיכנסו לחלון. כל הרשומות בחלון שוות לרשומה הנוכחית בשדות אלו.

- **Order** אופציונלי - לפי איזה סדר לסדר את הרשומות בתוך החלון. נשתמש אם נרצה להתייחס (בחלון למיקום היחסי של הרשימה (השוואה לרשומות לפני או אחרי

- **Frame**

בפריים נוכל להגדיר את הטווח הרלוונטי. ניתן להשתמש באופן דטמיניסטי רק כשהגדרנו סדר. מאפשרת יצירת תת קבוצה דינאמית שמבצעת חישובים בהשוואה לרשומות סמוכות (למשל חישוב סכום השורה הנוכחית עם השתיים שקדמו לה).

ניתן לייצר את החלון לפי סדר מסויים, בנוסף ניתן לסדר את השאלתה עצמה לפי סדר. אם הסדר בשני המקרים שונה, לעיתים זה מעיד על שגיאה.

בשיעור הסתכלנו על מטפורה של Window Function כחלון בבניין

קומה בבניין תהיה הרשומה (שורה בטבלה, Current record) שהחלון מופעל עליה, כך נדע איזה "נוף" (תוצאה של הפונקציה) נצפה לקבל. ה order יקביל להאם נעלה או נרד בקומות וה frame תקביל לאילו קומות נוספות נוכל לראות.

- Current record - קומה
- Name - מרכז, שמאל, ימין
- Partition

פחות עובד במטפורה, אולי כיוון

בבנין הזה השכנים מסתכלים אחד על השני, לא החוצה

- Order - האם עולים או יורדים
- Frame - אילו קומות שכנות ניתן לראות מקומה מסויימת



הנה דוגמה המחשבת ממוצע בחודשים האחרונים ומציגה את אופן הגדרת החלון

```
SELECT product, year, month, volume,  
avg(volume) OVER w AS rolling_3m_avg # Avg aggregation as the window  
function  
FROM sales  
WINDOW w AS ( # Name - the common name "w"  
PARTITION BY product # Partition - only other rows of the same product  
ORDER BY year, month # Order chronologically, defining before and after  
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW # Frame - 3 last months  
)  
ORDER BY product, year, month; # Order the anchor rows - First by product, later  
by the same timeline
```

פונקציות

יש לנו קבוצה של רשומות ואנחנו צריכים להחזיר ערך יחיד (כמו ב group by).

אנחנו יכולים להשתמש בפונקציות האגרגציה המוכרות לנו מ group by (count, avg (לדוגמה) ולהפעיל אותן על החלון.

בנוסף ניתן להפעיל פונקציות הייעודיות לחלון-

<u>CUME_DIST()</u>	Cumulative distribution value
<u>DENSE_RANK()</u>	Rank of current row within its partition, without gaps
<u>FIRST_VALUE()</u>	Value of argument from first row of window frame
<u>LAG()</u>	Value of argument from row lagging current row within partition
<u>LAST_VALUE()</u>	Value of argument from last row of window frame
<u>LEAD()</u>	Value of argument from row leading current row within partition
<u>NTH_VALUE()</u>	Value of argument from N-th row of window frame
<u>NTILE()</u>	Bucket number of current row within its partition.
<u>PERCENT_RANK()</u>	Percentage rank value
<u>RANK()</u>	Rank of current row within its partition, with gaps
<u>ROW_NUMBER()</u>	Number of current row within its partition

הפונקציות מחולקות לסוגים שונים:

מיקום אבסולוטי

ROW_NUMBER – מעניקה מספר סידורי רץ, ייחודי ורציף לכל שורה בחלון ללא קשר לכפל ערכים.

RANK – מעניקה דירוג לשורות, כאשר במקרה של שוויון הדירוגים הבאים ידלגו על מספרים ושני ערכים שווים יקבלו דירוג זהה.

DENSE_RANK – מעניקה דירוג רציף לשורות ללא דילוג על מספרים במקרה של שוויון.

מיקום באחוזים

CUME_DIST – מחשבת את המיקום היחסי המצטבר של השורה (ההסתברות או אחוז השורות שערך קטן או שווה לערך הנוכחי).

PERCENT_RANK – מחשבת את הדירוג היחסי של השורה באחוזים בטווח שבין 0 ל-1.

מיקום יחסי

FIRST_VALUE – שולפת את הערך מהשורה הראשונה בחלון.

LAG – שולפת ערך מעמודה בשורה קודמת (לפי מרחק מוגדר) בתוך אותו החלון.

LAST_VALUE – שולפת את הערך מהשורה האחרונה בחלון.

LEAD – שולפת ערך מעמודה בשורה הבאה (לפי מרחק מוגדר) בתוך אותו החלון.

NTH_VALUE – שולפת את הערך מהשורה ה-N המדויקת (לפי המיקום שנקבע) בתוך החלון.

NTILE – מחלקת את השורות לקבוצות שוות בגודלן ומחזירה את מספר הקבוצה (העשירון/רבעון) של השורה.

שימושים נפוצים בחלון:

- הסתכלות על ערכי חודש קודם/עוקב (לפני/אחרי) כמו למשל השוואת רווחים והפסדים.
- מדד מתגלגל / מדד נע (לדוגמה: ממוצע של 3 החודשים האחרונים)
- ערך ספציפי (לדוגמה: הערך ה-10 במקום ערך המקסימום)
- אחוזונים
- מיקום יחסי

דוגמאות

דוגמאות חשובות מהשיעור

חישוב עבור כל ז'אנר את כמות הסרטים המשויכים אליו, ואת ההסתברות להשתייך אליו:

```
select
genre
, sum(movies) over(partition by genre) AS genre_movies
, sum(movies) over(partition by genre)/sum(movies) over() as genre_probability
, sum(movies) over() as movies
from (
select
genre
, count(distinct movie_id) as movies
from
movies_genres
group by
genre
) as inSql
order by genre_movies desc;
```

PARTITION BY אומר ל-SQL לחלק את המידע לפי ז'אנרים. (SUM) יתבצע רק עבור השורות של אותו ז'אנר.

כאשר הסוגריים של ה-OVER ריקים, החלון הוא גלובלי - כלומר, הוא מחשב את סך כל הסרטים בכל הטבלה יחד.

השורה האמצעית מחלקת את סך הסרטים בז'אנר הנוכחי בסך הסרטים הכולל, וכך מפיקה את ההסתברות הז'אנר.

נרצה לחשב את סך התפקידים של כל שחקן, לדרג אותם בסדר יורד מהשחקן העסוק ביותר . בנוסף, למדוד: מספר סידורי רץ, דירוג בהשוואה לאחרים, חלוקה לעשירונים ומיקום באחוזון.

```
select
row_number() over w as num
, rank() over w as actor_rank
, percent_rank() over w as percentile
, ntile(10) over w as decile
, inSql.*
from
(
select
actor_id
, count(*) as actor_roles
from
roles
group by
actor_id
) as inSql
WINDOW w AS (ORDER BY actor_roles desc, actor_id)
;
```

שורת ה-WINDOW w AS מגדירה קיצור דרך לחלון (נקרא לו w) שמסודר לפי כמות תפקידים מהגבוה לנמוך.

כעת נוכל לקרוא לחלון W במקום להגדיר אותו מחדש בכל פעם.

row_number() over w - מעניק מספר סידורי רץ וייחודי לכל שורה (1, 2, 3...) ללא קשר לכפל ערכים.

rank() over w - מעניק דירוג. אם לשני שחקנים יש אותו מספר תפקידים בדיוק, הם יקבלו דירוג זהה (למשל מקום 4), והדירוג הבא אחריהם ידלג (למקום 6).

percent_rank() over w - מחשב את האחוזון (המיקום היחסי של השורה בטווח שבין 0 ל-1).

ntile(10) over w - מחלק את כל השורות בטבלה ל-10 קבוצות שוות בגודלן, ומסמן עבור כל שחקן באיזה עשירון הוא נמצא.

נרצה לייצר רשימת הסרטים של כל במאי לפי סדר כרונולוגי, להשוות בין הציון של הסרט הנוכחי לציון של הסרט הקודם והסרט הבא של אותו הבמאי. במטרה היא לראות אם הבמאי השתפר או נחלש מסרט לסרט.

```
select
director_id
year ,
m.rank ,
lag(m.rank) OVER w AS rank_lag ,
lead(m.rank) OVER w AS rank_lead ,
m.rank - lag(m.rank) OVER w AS lag_diff ,
m.rank - lead(m.rank) OVER w AS lead_diff ,
FROM
movies_directors as md
join
movies as m
on
md.movie_id = m.id
;(window w as (partition by director_id order by director_id, year
```

window w as - מגדיר חלון שמחולק לפי כל במאי בנפרד ומסודר כרונולוגית לפי שנת יציאת הסרט.

lag(m.rank) over w - שולף את הדירוג של הסרט הקודם של אותו במאי (השורה הקודמת בציר הזמן).

lead(m.rank) over w - שולף את הדירוג של הסרט הבא של אותו במאי (השורה הבאה בציר הזמן).

m.rank - lag(m.rank) over w - מחשב את ההפרש בין הסרט הנוכחי לקודם, כדי לראות אם הבמאי השתפר או נחלש בהשוואה לעבר.

איתור שגיאות בחלון

חשוב לזכור כי לא נוכל לראות ישירות את תוכן החלון (ולא ניתן להשתמש ב MySQL בטריקים כמו GROUP_CONCAT כדי להציג את התוכן שלו). לכן, מומלץ לבדוק את השאילתה על שורות ספציפיות לניסיון כדי לוודא שסדר החישוב הפנימי אכן עובד כצפוי. בנוסף, ניתן למיין את הרשומות ולשלוף את הרשומה הראשונה, שניה וכדומה.

הנה דוגמה להצצה לתוך החלון:

```
# Debug - looking into the window...

# MySQL currently does not support group concat to get all the window items
# However, for a fixed size window we can get them one by one

SELECT product, year, month, volume,
       avg(volume) OVER w AS rolling_3m_avg # Avg aggregation as the window
function
       , concat(product, '_', year, '_', month, '_', volume) as current_record
       , count(*) over w as window_records
       , NTH_VALUE(concat(product, '_', year, '_', month, '_', volume), 1) over w as
w1 # Look at the first record in the window (the current one)
       , NTH_VALUE(concat(product, '_', year, '_', month, '_', volume), 2) over w as
w2 # The second
       , NTH_VALUE(concat(product, '_', year, '_', month, '_', volume), 3) over w as
w3 # The third
       , NTH_VALUE(concat(product, '_', year, '_', month, '_', volume), 4) over w as
w4 # No such item in window, always null

FROM sales

WINDOW w AS ( # Name - the common name "w"

    PARTITION BY product # Partition - only other rows of the same product

    ORDER BY year, month # Order chronologically, defining before and after

    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW # Frame - 3 last
months
)

ORDER BY product, year, month; # Order the anchor rows - First by product,
later by the same timeline
```


חלק 2 - דיון על תכנון מודל

מחקר אג'ילי

מומלץ לבצע את הפרויקט באמצעות מתודולוגיית מחקר אג'ילי.

- בצעו צעד קטן, קבלו משוב והמשיכו הלאה.
- במקום לצלול לעומק של מודל בודד, בנו גרסה פשוטה של כל המודלים, העריכו את הביצועים שלהם והשתמשו במידע זה כדי להחליט איפה כדאי להשקיע יותר זמן.
- חלקכם מתכננים להעשיר את הנתונים - זה מעולה ומומלץ. עם זאת חשוב לא להרחיק לכת עם שימוש בנתונים שלא כלולים בדאטה שלנו ונאספים ממקורות חיצוניים (למשל מי חברת ההפקה), על מנת להבטיח שניתן יהיה למצוא את המידע הרלוונטי לכל הסרטים.
- רובכם תכננתם מודלים פשוטים המבוססים על טבלה בודדת וכוללים מעט פרמטרים. אלו מודלים מעולים להתחיל איתם - הם מהירים ליישום וקל להבין את השגיאות שלהם.

מדד מגדלור

ישנם מדדים שאפשר לקחת לקיצוניות (לדוגמה, להגיע ל-100% Recall פשוט על ידי זה שממליצים על הכל). אם אתם עושים אופטימיזציה למדד כזה, הוסיפו אילוץ (למשל: Precision מינימלי של X%, והסבירו מדוע בחרתם בערך של X).

- השתמשו במדד שתוכלו ליישם בקלות בתוך בסיס הנתונים (לדוגמה, מדד כמו precision 20 עשוי לדרוש השקעה רבה ליישום).
- כבר יש לכם יישום מוכן של [Confusion Matrix](#).

מדדים במודלים

- אתם יכולים להשתמש בכל מדד כדי לשפר את המודל שלכם.
- אינכם חייבים להגביל את עצמכם רק לחישוב Counts ומדד ג'אקארד שהוצגו בכיתה.
- לצוות שמתכנן להשתמש בהחלקת לפלס - כל הכבוד!
- כדי לדעת מהם ביצועי המודל עלינו להעריך אותו. יחד עם זאת, פיתוח אינטואיציה יכול לחסוך לכם המון זמן.
- לכל סרט יש כנראה רק מעט המלצות טובות באמת. אם המודל שלכם מתאים לו סרטים רבים מדי (למשל: בגלל שנים קרובות או אותו ז'אנר), תקבלו Precision נמוך מאוד.
- כשמתאימים סרטים לפי שחקנים: שחקן שהופיע בשני סרטים בלבד יכול להעיד על קשר חזק מאוד, או לחילופין על שגיאה מקרית.
- שחקן שהופיע בסרטים רבים עשוי להיות כוכב גדול - מה שיכול להעיד על קשר חזק, או לחילופין להיות גורם שמחבר בין סרטים ללא שום קשר אמיתי ביניהם.
- כאשר יש לנו אינטואיציות סותרות, הערכות מהירות בהחלט עוזרות.
- לרוב הסרטים אין יותר מדי ז'אנרים. שימוש במדד ג'אקארד על מספר הז'אנרים התואמים יחזיר את אותן התוצאות ברוב המקרים, מה שהופך אותו לפחות יעיל.
- הסוד לביצועים גבוהים טמון בדרך כלל במניפולציה של הנתונים. נסו להבין היכן המודל שלכם מניח הנחות לא מציאותיות (לדוגמה: שכל התפקידים הם בעלי חשיבות דומה; עדיף להתעלם מתפקידי "בתפקיד עצמו" מאשר להשתמש בטקסט "X בתפקיד עצמו"; או התייחסות ל-"Policeman" ו-"Police officer" כתפקידים שונים). טיפול במקרים כאלה הוא בדרך כלל משתלם מאוד.

סינון שיתופי

- סינון שיתופי ("אנשים שאהבו את X , אהבו גם את Y ") מאפשר לכם לדלג על המסלולים הקשיחים של במאים, שחקנים וכדומה. מומלץ מאוד שיהיה לכם לפחות מודל אחד כזה.

אגרגציה

- למרות שאתם יכולים לשקלל את ה"הצבעות" של המודל שלכם פשוט באמצעות count/sum/avg, זהו בזבוז של ביצועים. מומלץ ללמוד את המשקל של כל מודל (למשל באמצעות רגרסיה ליניארית), אך זה אינו חובה. עם זאת, הערכת המשקל עבור כל מודל היא מועילה.
- בנוסף, אתם לא חייבים להגביל את עצמכם להצבעות בינאריות (כן/לא). המודל שלכם יכול לספק רמות ביטחון - לדוגמה, שיתוף של 6 שחקנים הוא התאמה חזקה יותר מאשר שיתוף של 3 שחקנים.